

Module 05 Lab - Data Preparation **Objective:** To learn and apply the most common data preparation techniques. Raw data is rarely ready for a machine learning model.

- ✓ This process, also called preprocessing, is one of the most critical steps in the entire ML workflow. **In this lab, you will write more of the code.** Read the explanations and then complete the tasks in the code cells.

- ✓ Part 1: Setup and Initial Look We will continue using the Titanic dataset because it has the exact problems we need to solve: missing values and non-numeric data.

```
import pandas as pd
import numpy as np
# Load the dataset
df = pd.read_csv('https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv')
# Let's look at the missing values
print("--- Missing Values Before Cleaning ---")
print(df.isnull().sum())
```

```
--- Missing Values Before Cleaning ---
PassengerId      0
Survived          0
Pclass           0
Name             0
Sex              0
Age             177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin           687
Embarked         2
dtype: int64
```

Part 2: Handling Missing Values (Imputation) **Concept:** Most machine learning models cannot handle missing values (**NaN**). We must deal with them. Dropping the rows is an option, but you lose data. A better way is **imputation**, which means filling in the missing values with a calculated guess. Common imputation strategies:

- * **Mean:** Fill with the average value. Good for normally distributed data.
- * **Median:** Fill with the middle value. Better for skewed data or data with outliers (like **Fare**).
- * **Mode:** Fill with the most frequent value. Used for categorical data.

- ✓ Task 1: Impute the 'Age' Column The 'Age' column is missing many values. Since age can be skewed (e.g., by a few very old passengers), using the **median** is a robust choice. **Your Task:** Calculate the median of the 'Age' column and use the `.fillna()` method to replace the missing values.

```
# --- ENTER YOUR CODE HERE ---

# Calculate the median
median_age = df['Age'].median()
# Fill missing values
df['Age'] = df['Age'].fillna(median_age)
# Verify
print("Missing values in 'Age' after imputation:")
9
print(df['Age'].isnull().sum())
```

```
Missing values in 'Age' after imputation:
0
```

Part 3: Encoding Categorical Features**Concept:** Machine learning models are mathematical, so they need numbers, not text. We need to convert categorical columns (like 'Sex' and 'Embarked') into a numerical format. The most common method is **One-Hot Encoding**. One-Hot Encoding takes a column with **N** categories and turns it into **N** new columns, each with a **1** or **0**. For example, the 'Sex' column (**male**, **female**) becomes two new columns: **Sex_male** and **Sex_female**. Pandas has a convenient function called **pd.get_dummies()** that does this for us.

Task 2: One-Hot Encode Categorical Columns**Your Task:** Use **pd.get_dummies()** to encode the 'Sex' and 'Embarked' columns. Make sure to drop the original columns after encoding.

```
# --- ENTER YOUR CODE HERE ---
# # 1. Create dummy variables for 'Sex' and 'Embarked'
encoded_df = pd.get_dummies(
    df,
    columns=['Sex', 'Embarked'],
    drop_first=True
)

# 2. Display the first few rows
print(encoded_df.head())
```

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Age	SibSp	Parch	\
0	Braund, Mr. Owen Harris	22.0	1	0	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	38.0	1	0	
2	Heikkinen, Miss. Laina	26.0	0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	35.0	1	0	
4	Allen, Mr. William Henry	35.0	0	0	

	Ticket	Fare	Cabin	Sex_male	Embarked_Q	Embarked_S
0	A/5 21171	7.2500	NaN	True	False	True
1	PC 17599	71.2833	C85	False	False	False
2	STON/O2. 3101282	7.9250	NaN	False	False	True
3	113803	53.1000	C123	False	False	True
4	373450	8.0500	NaN	True	False	True

Part 4: Feature Scaling**Concept:** Many models are sensitive to the scale of the features. For example, **Age** (from 0-80) and **Fare** (from 0-512) are on very different scales. This can cause the model to incorrectly believe that **Fare** is a more important feature simply because its values are larger. **Feature Scaling** solves this by putting all features on a similar scale. A common method is **Standardization** (**StandardScaler** in scikit-learn), which rescales the data to have a mean of 0 and a standard deviation of 1. **Important:** You only scale your numerical features, not your target variable or your newly encoded categorical columns.

Task 3: Scale the 'Age' and 'Fare' Columns**Your Task:** Use **StandardScaler** from **sklearn.preprocessing** to scale the 'Age' and 'Fare' columns.

```
from sklearn.preprocessing import StandardScaler
# 1. Create an instance of the StandardScaler
scaler = StandardScaler()

# 2. Select the columns to scale
columns_to_scale = ['Age', 'Fare']

# 3. Fit the scaler to the data and transform it
```

```
encoded_df[colums_to_scale] = scaler.fit_transform(encoded_df[colums_to_scale])
```

```
# 4. Display the first few rows to see the scaled data
print(encoded_df.head())
```

```
 PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

      Name      Age  SibSp  Parch  \
0      Braund, Mr. Owen Harris  -0.565736    1    0
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  0.663861    1    0
2      Heikkinen, Miss. Laina  -0.258337    0    0
3  Futrelle, Mrs. Jacques Heath (Lily May Peel)  0.433312    1    0
4      Allen, Mr. William Henry  0.433312    0    0

      Ticket    Fare  Cabin  Sex_male  Embarked_Q  Embarked_S
0  A/5 21171  -0.502445   NaN      True      False      True
1  PC 17599   0.786845   C85     False      False      False
2  STON/O2. 3101282  -0.488854   NaN     False      False      True
3   113803   0.420730  C123     False      False      True
4   373450  -0.486337   NaN      True      False      True
```

 **Knowledge CheckInstructions:** Answer the following questions in this markdown cell.1.

****Why is it often better to impute missing values with the median instead of the mean?****2.

- ✓ **Explain in your own words what One-Hot Encoding does and why it is necessary.**3. **Would you need to apply Feature Scaling to a Decision Tree model? Why or why not? (Hint: Think about how a Decision Tree makes its splits).[ENTER YOUR ANSWERS HERE]**

1. Why is it often better to impute missing values with the median instead of the mean?

The median is less affected by extreme values (outliers). If the data contains unusually high or low values, the mean can be pulled away from the center of the data, while the median remains more representative of a typical value.

2. Explain in your own words what One-Hot Encoding does and why it is necessary.

One-Hot Encoding converts categorical data (such as "Male/Female" or "Embarked" locations) into numerical columns that a machine learning model can understand. It creates a separate column for each category and uses 1 or 0 to indicate whether that category applies. This is necessary because most machine learning algorithms work with numbers, not text.

3. Would you need to apply Feature Scaling to a Decision Tree model? Why or why not?

No, feature scaling is usually not necessary for a Decision Tree. Decision Trees make decisions by splitting data based on feature values and thresholds, not by measuring distances between points. Therefore, the scale of the features does not affect how the tree chooses its splits.